

You ship pieces. Nothing carries the release.

Images, charts, SBOMs, configs — each signed differently, if at all. No identity for the release itself.

Open Component Model — open source, NeoNephos Foundation.

DIAGNOSIS

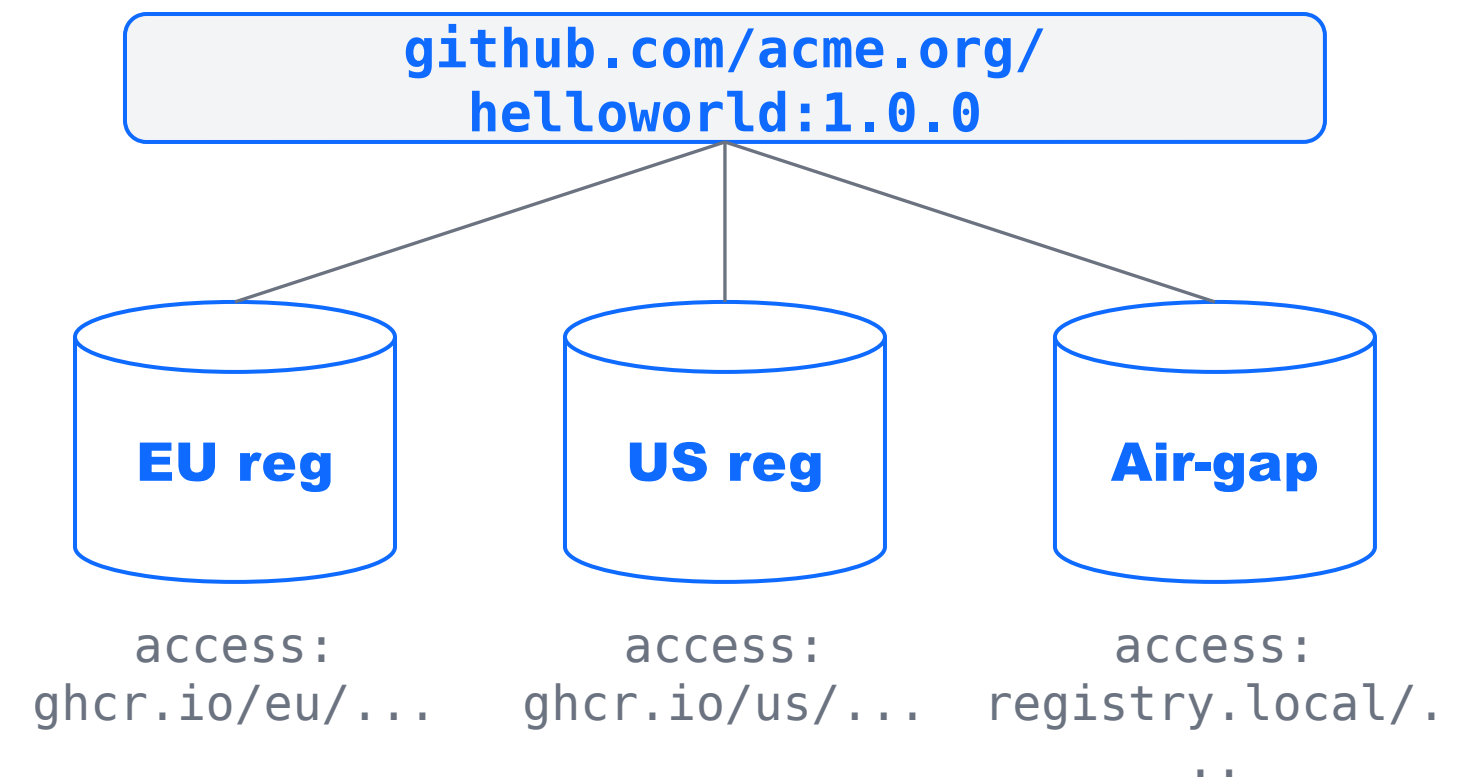
In every existing tool, identity is bound to location.

- **OCI image** — identified by registry/repo:tag. Mirror it; every downstream reference is invalidated.
- **Helm chart** — identified by repo URL + name + version. Mirror the repo; pulls fail.
- **SBOM** — linked to its subject by file path or naming convention. Move the artifact; the link dangles.

THE HINGE

Identity that travels with the artifact.

- **Coordinates** — name and version of the component. Globally unique. Location-agnostic.
- **Digest** — every resource inside the component carries a content hash. Computed once.
- **Access** — where the resource currently lives. Rewritten on transfer. Digest stays.



WHERE OCM SITS

One component wraps every artifact, signed once.

ANY FORMAT

Helm, OCI, SBOM, npm, binaries.
Each becomes a resource inside the component.

ANY LOCATION

Coordinates travel.
The component carries its name across registries.

ONE SIGNATURE

Covers every digest in the component.
Survives transport.

CONSTRUCTOR

What you write.

```
components:
- name: github.com/acme.org/helloworld
  version: 1.0.0
  provider:
    name: acme.org
  resources:
    - name: mylocalfile
      type: blob
      input:                # Embed by value
        type: File/v1
        path: ./my-local-resource.txt
    - name: image
      type: ociImage
      version: 1.0.0
      access:                # Reference external artifact
        type: OCIImage/v1
        imageReference: ghcr.io/stefanprodan/podinfo:6.9.1
  componentReferences:      # Compose other components
    - name: notes
      componentName: acme.org/sovereign/notes
      version: 1.0.0
```

input: by value

Embed bytes at pack time.

access: by reference

Resolve external artifact at pack time.

componentReferences: compose

Point to other components by name + version.

DESCRIPTOR

What gets signed and travels.

```
component:                                # (fields trimmed)
  name: github.com/acme.org/helloworld
  version: 1.0.0
  resources:
    - name: mylocalfile
      access:                               # excluded
        type: LocalBlob/v1
        localReference: sha256:70a2577d7b...
      digest:                               # input to hash
        hashAlgorithm: SHA-256
        value: 70a2577d7b649574cbbba99a...
    - name: image
      access:
        imageReference: ghcr.io/.../podinfo@sha256:8fa569...
      digest:
        value: 262578cde928d5c9eba3bce0...
  signatures:
    - name: acme-release-key
      digest:                               # of the descriptor
        value: 91dd197868907487e62872695db1...
      signature:
        algorithm: RSASSA-PSS
        value: <hex-encoded signature>
```

access: excluded

Rewritten on every transfer.

digest: content identity

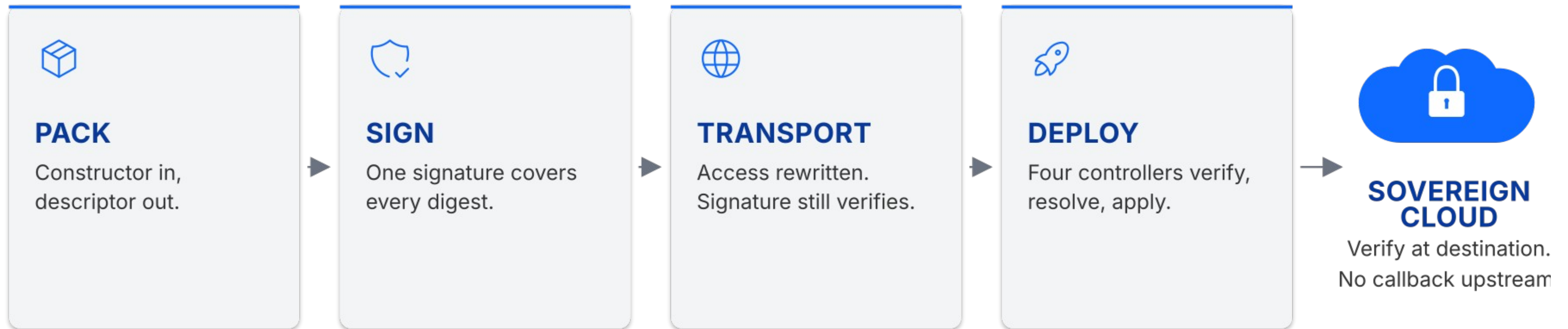
Input to the descriptor hash.

signature: one hash

Over the canonicalized descriptor.

FOUR VERBS, ONE COMPONENT

Pack · Sign · Transport · Deploy.



COMPOSE

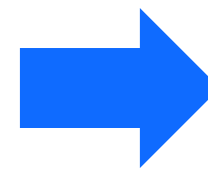
Components compose into products.

Leaf components carry resources — images, charts, configs, SBOMs.

A product or release component composes other components by **name and version**. One unit, transferable, signable end-to-end.

LEAF

```
# leaf component
component:
  name: acme.org/sovereign/notes
  version: 1.0.0
  resources:
    - name: image
      type: ociImage
      access:
        type: OCIImage/v1
        imageReference: ghcr.io/.../notes:1.0.0
    - name: chart
      type: helmChart
      access: { type: helmChart/v1, ... }
```



PRODUCT

```
# product / release component
component:
  name: acme.org/sovereign/product
  version: 1.0.0
  componentReferences:
    - name: notes
      componentName: acme.org/sovereign/notes
      version: 1.0.0
    - name: postgres
      componentName: acme.org/sovereign/postgres
      version: 1.0.0
  # no resources of its own – pure composition
```


SIGN

One signature shape. Three trust models.

RSA

Your existing PKI.
Keys you already rotate.

GPG

OpenPGP keys.
Familiar trust model, ASCII-armored.

SIGSTORE

Keyless via OIDC.
Identity, not long-lived keys.

TRANSPORT

Three patterns. One command.

REGISTRY → REGISTRY

Promote across stages.

Source registry to target registry.

REGISTRY → CTF

Export to a local archive.

Hand-carry across the boundary.

CTF → REGISTRY

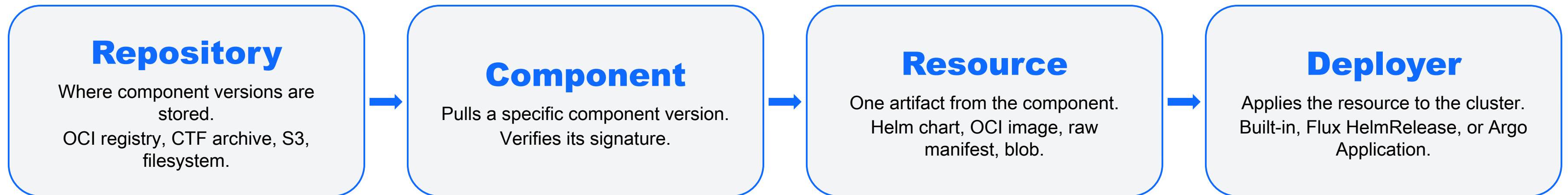
Air-gap import.

Verify on arrival. No callback to source.

DEPLOY

Four controllers verify and apply the component.

Four Kubernetes Custom Resources, one per layer.



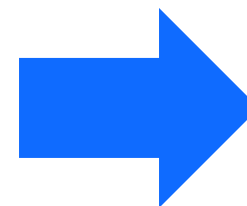
DAY 2

Bump the product. Everything follows.

To upgrade, raise the product version. **The references follow.** Notes patched from 1.0.0 to 1.1.0; postgres untouched.

DAY 1

```
# day 1 – product 1.0.0
component:
  name: acme.org/sovereign/product
  version: 1.0.0
  componentReferences:
    - name: notes
      version: 1.0.0
    - name: postgres
      version: 1.0.0
```



DAY 2

```
# day 2 – product 1.1.0
component:
  name: acme.org/sovereign/product
  version: 1.1.0
  componentReferences:
    - name: notes
      version: 1.1.0
    - name: postgres
      version: 1.0.0
```

ADOPTION

Two paths to a first OCM component.

FROM ZERO — CLI

Pack one component. Sign it.

Air-gap CTF round-trip.

Verify on the other side.

Thirty minutes. One afternoon.

ON YOUR CLUSTER — CONTROLLERS

Helm-install the OCM controllers.

Point them at your registry.

Apply a Component resource — verified and reconciling.

Same on OpenControlPlane (one openMCP resource).

WHAT'S SHARP

Three honest edges.

- **Helm chart access** — not fully resolved at pack time. Use `--skip-reference-digest-processing` for the Helm/v1 access type.
- **Transfer defaults** — copies only the descriptor. For air-gap, pass `--copy-resources` so the bytes travel too.
- **Controllers are v1alpha1** — the CRD surface can move. Pin minor versions in your platform installs.

Ship the release as one unit.

Try it — ocm.software

Build with us — github.com/open-component-model

Talk to us — community channels on the website